

Lecture Notes: Recurrent Neural Networks (RNNs) for Autoregressive Modeling in Time Series

1 Introduction

Recurrent Neural Networks (RNNs) are widely used for sequential data due to their ability to retain information over time. In this document, we review the fundamentals of RNNs, covering input and hidden dimensions, the mechanics of autoregressive modeling, and a detailed code example in PyTorch. This discussion aims to solidify concepts and clarify parameter roles within RNN architectures.

2 Core Concepts of RNNs

An RNN processes sequences step-by-step, retaining a hidden state that acts as a form of memory. At each time step t , the RNN updates its hidden state h_t based on the input X_t and the previous hidden state h_{t-1} . This allows RNNs to model temporal dependencies in data.

2.1 Input and Hidden Dimensions

Key dimensions in RNNs include:

- **Input dimension** (*input_size*): Number of features per time step in the input sequence.
- **Hidden dimension** (*hidden_dim*): Number of neurons in the RNN's hidden layer, determining the capacity of the RNN to retain information over time.
- **Output dimension** (*output_size*): Typically the number of features we want the RNN to predict, matching the input feature space in autoregressive models.
- **Number of layers** (*n_layers*): The number of stacked RNN layers. More layers can capture more complex patterns but may increase training complexity.

2.2 Autoregressive Modeling for Time Series

In autoregressive modeling, the RNN predicts the next time step based on the current sequence. Training uses a shifted target series, where X_{t+1} is the label for X_t . For forecasting, the last predicted value becomes the input for the next time step iteratively.

2.3 Why Use RNNs for Autoregression?

The autoregressive approach allows the RNN to forecast sequences over multiple time steps. By iteratively predicting and feeding predictions back into the RNN, it generates a series of outputs based on learned temporal patterns.

3 Detailed Code Walkthrough

The following code defines an RNN class for autoregressive modeling.

```
class RNN(nn.Module):
    def __init__(self, input_size, output_size, hidden_dim, n_layers, sigma):
        super().__init__()
        self.hidden_dim = hidden_dim
        self.input_size = input_size
        self.output_size = output_size
        self.sigma = torch.Tensor(np.array(sigma))
        self.rnn = nn.RNN(input_size=input_size, hidden_size=hidden_dim,
                           num_layers=n_layers, nonlinearity='relu', batch_first=True)
        self.fc1 = nn.Linear(hidden_dim, output_size)
        self.fc2 = nn.Linear(hidden_dim, output_size)

    def forward(self, x, h0=None):
        batch_size = x.size(0)
        seq_length = x.size(1)
        r_out, hidden = self.rnn(x, h0)
        r_out = r_out.reshape(-1, self.hidden_dim)
        mean = self.fc1(r_out)
        variance = torch.exp(self.fc2(r_out)) + torch.ones(mean.shape) * self.sigma
        noise = torch.randn_like(mean) * torch.sqrt(variance)
        sample = mean + noise
        sample = sample.reshape([-1, seq_length, int(self.output_size)])
        mean = mean.reshape([-1, seq_length, int(self.output_size)])
        variance = variance.reshape([-1, seq_length, int(self.output_size)])
        return mean, variance, hidden, sample
```

3.1 Explanation of Code Components

3.1.1 RNN Layer Definition

The line

```
self.rnn = nn.RNN(input_size=input_size, hidden_size=hidden_dim,  
                  num_layers=n_layers, nonlinearity='relu', batch_first=True)
```

defines an RNN with the specified input size, hidden size, and number of layers.

Output of the RNN layer:

- **r_out**: Contains the hidden states for each time step, shape (batch_size, seq_length, hidden_dim).
- **hidden**: The last hidden state for each layer, useful for forecasting.

3.1.2 Reshaping for Processing

The line

```
r_out = r_out.reshape(-1, self.hidden_dim)
```

flattens **r_out** to (batch_size × seq_length, hidden_dim), allowing processing by the linear layers.

3.1.3 Calculating Mean and Variance

The linear layers **fc1** and **fc2** estimate the mean and log-variance. The variance is regularized by adding a small constant σ :

$$\text{variance} = \exp(\text{fc2}(r_out)) + \sigma \quad (1)$$

3.1.4 Generating Sample with Noise

Noise is added to the mean to create a sample for each output:

$$\text{sample} = \text{mean} + \text{noise} \quad (2)$$

where noise is generated as a Gaussian sample scaled by the standard deviation.

3.2 Autoregressive Prediction Flow

1. Feed in sequence chunks up to T_{train} during training.
2. Use the last hidden state as the initial state for forecasting.
3. Predict iteratively from T_{train} to T by using the last output as the next input.

4 Additional Notes

In autoregressive prediction, errors can accumulate due to iterative inputs, known as **exposure bias**. Techniques like **teacher forcing** address this by feeding true values during training.

5 Conclusion

These notes covered the architecture, parameters, and use of RNNs for autoregressive prediction. The concepts here should serve as a foundational reference for understanding and implementing RNN-based models for sequential data.